Assign

Your Name Here

Contents

Load dataset . . . 1
Feature: Direction . . . 1
- Modeling . . . 1
- MSE of direction . . . 2
- Bias of direction . . . 2
- Confidence Intervals for 'direction' . . . 3
- Likelihood Ratio Test . . . 3
- Wald Test . . . 4
- Bias Plot . . . 4

Feature: Section . . . 4
- Modeling . . . 4
- MSE of section . . . 5
- Bias of section . . . 5
- Confidence Intervals for 'section' . . . 5
- Likelihood Ratio Test . . . 6
- Wald Test . . . 6
- Bias Plot of section . . . 6

Feature: ERROR . . . 7
- Modeling . . . 7
- MSE and bias . . . 8
- Likelihood ratio test . . . 8
- Wald test . . . 8
- Plot bias . . . 9


```
library(readxl)
library(nnet)
library(MASS)
```

Load dataset

```
data <- read_xlsx("dataset.xlsx")
```

Feature: Direction

Modeling

```
data$Direction <- as.factor(data$Direction)
data$Section <- as.factor(data$Section)


direction_model <- multinom(Direction ~ ERROR, data = data)
```

```
## # weights:  3 (2 variable)
## initial  value 166.355323
## final  value 166.328732
## converged
```

```
direction_pred <- predict(direction_model, type = "probs")


direction_params <- coef(direction_model)


log_likelihood <- function(params, data) {
  model <- multinom(direction ~ error, data = data, weights = exp(params))
  return(-sum(logLik(model)))
}


laplace_approximation <- function(model, data) {

  params <- coef(model)

  hessian <- solve(vcov(model))

  variance <- ginv(hessian)

  return(list(params = params, variance = variance))
}

direction_laplace <- laplace_approximation(direction_model, data)

direction_params <- direction_laplace$params
direction_variance <- direction_laplace$variance
```

MSE of direction

```
direction_actual <- model.matrix(~ Direction - 1, data = data)
mse_direction <- -mean(rowSums(direction_actual * log(direction_pred)))


cat("MSE (direction):", mse_direction, "\n")
```

```
## MSE (direction): 0.6932585
```

Bias of direction

```
direction_pred_class <- predict(direction_model, type = "class")


bias_direction <- mean(direction_pred_class != data$Direction)


cat("Bias (direction):", bias_direction, "\n")
```

```
## Bias (direction): 0.5
```

Confidence Intervals for 'direction'

```
conf_level <- c(0.95, 0.98, 0.99)
alpha <- 1 - conf_level
params <- coef(direction_model)
se <- sqrt(diag(vcov(direction_model)))

conf_intervals <- sapply(alpha, function(a) {
  z <- qnorm(1 - a / 2)
  ci_lower <- params - z * se
  ci_upper <- params + z * se
  return(data.frame(ci_lower, ci_upper))
}, simplify = FALSE)


names(conf_intervals) <- paste(conf_level * 100, "% CI", sep = "")
conf_intervals
```

```
## $`95% CI`
##               ci_lower  ci_upper
## (Intercept) -0.5944938 0.7397278
## ERROR       -0.5188591 0.4096295
##
## $`98% CI`
##               ci_lower  ci_upper
## (Intercept) -0.7191994 0.8644335
## ERROR       -0.6056422 0.4964126
##
## $`99% CI`
##               ci_lower  ci_upper
## (Intercept) -0.8041152 0.9493492
## ERROR       -0.6647353 0.5555057
```

Likelihood Ratio Test

```
direction_counts <- table(data$Direction)
prob_vector <- rep(1/4, length(direction_counts))
log_likelihood_null <- sum(dmultinom(direction_counts, prob = prob_vector, log = TRUE))

log_likelihood_alternative <- sum(logLik(direction_model))

lrt_direction <- -2 * (log_likelihood_null - log_likelihood_alternative)
p_value_lrt_direction <- pchisq(lrt_direction, df = length(coef(direction_model)), lower.tail = FALSE)

cat("Likelihood Ratio Test statistic (direction):", lrt_direction, "\n")
```

```
## Likelihood Ratio Test statistic (direction): -326.7232
```

```
cat("p-value for LRT (direction):", p_value_lrt_direction, "\n")
```

```
## p-value for LRT (direction): 1
```

Wald Test

```
wald_direction <- (params / se)^2
p_value_wald_direction <- pchisq(wald_direction, df = 1, lower.tail = FALSE)

cat("Wald Test statistic (direction):", sum(wald_direction), "\n")
```

```
## Wald Test statistic (direction): 0.09868208
```

```
cat("p-value for Wald Test (direction):", p_value_wald_direction, "\n")
```

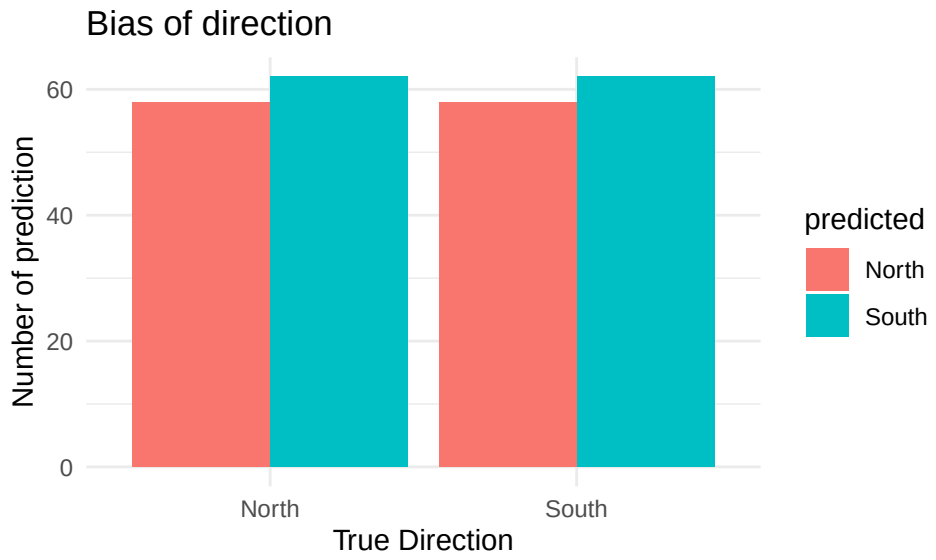
```
## p-value for Wald Test (direction): 0.8310556 0.8176452
```

Bias Plot

```
library(ggplot2)

direction_pred_df <- data.frame(actual = data$Direction, predicted = direction_pred_class)

ggplot(direction_pred_df, aes(x = actual, fill = predicted)) +
  geom_bar(position = "dodge") +
  labs(title = "Bias of direction", x = "True Direction", y = "Number of prediction") +
  theme_minimal()
```



Feature: Section

Modeling

```
section_model <- multinom(Section ~ ERROR, data = data)
```

```
## # weights:  15 (8 variable)
## initial  value 386.265099
## iter  10 value 380.864268
## final  value 380.864220
## converged
```

```r
section_pred <- predict(section_model, type = "probs")

section_params <- coef(section_model)

direction_laplace <- laplace_approximation(direction_model, data)

direction_params <- direction_laplace$params
direction_variance <- direction_laplace$variance
```

MSE of section

```r
section_actual <- model.matrix(~ Section - 1, data = data)
mse_section <- -mean(rowSums(section_actual * log(section_pred)))


cat("MSE (Section):", mse_section, "\n")
```

```
## MSE (Section): 1.586934
```

Bias of section

```r
section_pred_class <- predict(section_model, type = "class")


bias_section <- mean(section_pred_class != data$Section)


cat("Bias (Section):", bias_section, "\n")
```

```
## Bias (Section): 0.7625
```

Confidence Intervals for 'section'

```r
conf_level <- c(0.95, 0.98, 0.99)
alpha <- 1 - conf_level
params <- coef(section_model)
se <- sqrt(diag(vcov(section_model)))

conf_intervals <- sapply(alpha, function(a) {
  z <- qnorm(1 - a / 2)
  ci_lower <- params - z * se
  ci_upper <- params + z * se
  return(data.frame(ci_lower, ci_upper))
}, simplify = FALSE)


names(conf_intervals) <- paste(conf_level * 100, "% CI", sep = "")
conf_intervals
```

```
## $`95% CI`
##           X.Intercept.     ERROR X.Intercept..1    ERROR.1
## section B   0.50456046 -2.348383       2.708691 -0.1439674
## section C  -0.33791901 -1.050807       1.263085  0.3987525
```

```
## section D  -0.62948949 -1.432710       1.575280  0.7656091
## section E  -0.01082987 -1.249755       1.437810  0.2237236
##
## $`98% CI`
##           X.Intercept.     ERROR X.Intercept..1   ERROR.1
## section B    0.2985470 -2.554423       2.914704 0.0620727
## section C   -0.4875601 -1.186293       1.412726 0.5342385
## section D   -0.8355627 -1.638181       1.781353 0.9710794
## section E   -0.1462298 -1.387477       1.573210 0.3614452
##
## $`99% CI`
##           X.Intercept.     ERROR X.Intercept..1   ERROR.1
## section B    0.1582665 -2.694721       3.054985 0.2023714
## section C   -0.5894550 -1.278549       1.514621 0.6264948
## section D   -0.9758839 -1.778091       1.921675 1.1109901
## section E   -0.2384276 -1.481255       1.665407 0.4552239
```

Likelihood Ratio Test

```
section_counts <- table(data$Section)
prob_vector <- rep(1/4, length(section_counts))
log_likelihood_null <- sum(dmultinom(section_counts, prob = prob_vector, log = TRUE))

log_likelihood_alternative <- sum(logLik(section_model))

lrt_section <- -2 * (log_likelihood_null - log_likelihood_alternative)
p_value_lrt_section <- pchisq(lrt_section, df = length(coef(section_model)), lower.tail = FALSE)

cat("Likelihood Ratio Test statistic (section):", lrt_section, "\n")
```

```
## Likelihood Ratio Test statistic (section): -740.4849
```

```
cat("p-value for LRT (section):", p_value_lrt_section, "\n")
```

```
## p-value for LRT (section): 1
```

Wald Test

```
wald_section <- (params / se)^2
p_value_wald_section <- pchisq(wald_section, df = 1, lower.tail = FALSE)

cat("Wald Test statistic (section):", sum(wald_section), "\n")
```

```
## Wald Test statistic (section): 21.78549
```

```
cat("p-value for Wald Test (section):", p_value_wald_section, "\n")
```

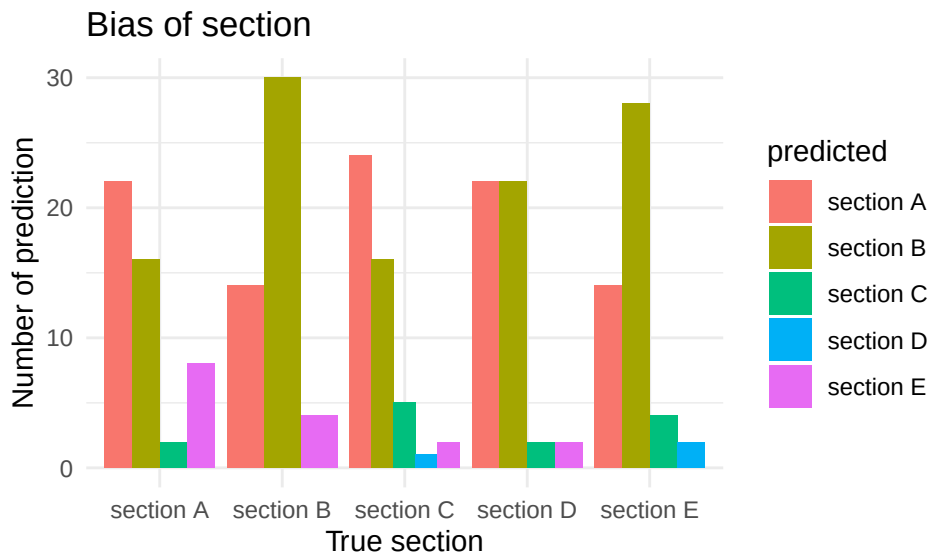
```
## p-value for Wald Test (section): 0.004272649 0.2573836 0.4004739 0.05352523 0.02669361 0.3779665 0.5
```

Bias Plot of section

```
library(ggplot2)

section_pred_df <- data.frame(actual = data$Section, predicted = section_pred_class)
```

```r
ggplot(section_pred_df, aes(x = actual, fill = predicted)) +
  geom_bar(position = "dodge") +
  labs(title = "Bias of section", x = "True section", y = "Number of prediction") +
  theme_minimal()
```



Feature: ERROR

Modeling

```r
error_model <- lm(ERROR ~ Direction + Section, data = data)
summary(error_model)
```

```
##
## Call:
## lm(formula = ERROR ~ Direction + Section, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.09413 -0.36337 -0.06065  0.32296  1.54888
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        1.47752    0.08541  17.299  < 2e-16 ***
## DirectionSouth    -0.01623    0.06974  -0.233  0.81616
## Sectionsection B -0.34180    0.11026  -3.100  0.00217 **
## Sectionsection C -0.10027    0.11026  -0.909  0.36410
## Sectionsection D -0.10249    0.11026  -0.929  0.35361
## Sectionsection E -0.15427    0.11026  -1.399  0.16310
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5402 on 234 degrees of freedom
## Multiple R-squared:  0.04295,    Adjusted R-squared:  0.0225
## F-statistic:   2.1 on 5 and 234 DF,  p-value: 0.06619
```

MSE and bias

```
error_pred <- predict(error_model)
mse_error <- mean((data$ERROR - error_pred)^2)
bias_error <- mean(data$ERROR - error_pred)

cat("MSE (ERROR):", mse_error, "\n")
```

```
## MSE (ERROR): 0.2844984
```

```
cat("BIAS (ERROR):", bias_error, "\n")
```

```
## BIAS (ERROR): -5.738185e-17
```

Likelihood ratio test

```
log_likelihood_null <- sum(dnorm(data$ERROR, mean(data$ERROR), sd(data$ERROR), log = TRUE))

log_likelihood_alternative <- sum(logLik(error_model))

lrt_statistic <- -2 * (log_likelihood_null - log_likelihood_alternative)
lrt_p_value <- pchisq(lrt_statistic, df = length(coef(error_model)) - 1, lower.tail = FALSE)


cat("Likelihood Ratio Test statistic (ERROR):", lrt_statistic, "\n")
```

```
## Likelihood Ratio Test statistic (ERROR): 10.53871
```

```
cat("p-value for LRT (ERROR):", lrt_p_value, "\n")
```

```
## p-value for LRT (ERROR): 0.06133308
```

Wald test

```
error_params <- coef(error_model)
error_se <- sqrt(diag(vcov(error_model)))

wald_statistic <- (error_params / error_se)^2
wald_p_value <- pchisq(wald_statistic, df = 1, lower.tail = FALSE)

wald_statistic
```

```
##      (Intercept)   DirectionSouth Sectionsection B Sectionsection C
##     299.26321752       0.05417431       9.60909503       0.82692301
## Sectionsection D Sectionsection E
##       0.86390470       1.95756576
```

```
wald_p_value
```

```
##      (Intercept)   DirectionSouth Sectionsection B Sectionsection C
##     4.767515e-67     8.159526e-01     1.936160e-03     3.631638e-01
## Sectionsection D Sectionsection E
##     3.526483e-01     1.617739e-01
```

```
conf_levels <- c(0.95, 0.98, 0.99)
alpha <- 1 - conf_levels
error_params <- coef(error_model)
```

```r
error_se <- sqrt(diag(vcov(error_model)))

for (a in alpha) {
  z <- qnorm(1 - a / 2)
  ci_lower <- error_params - z * error_se
  ci_upper <- error_params + z * error_se
  conf_int <- data.frame(ci_lower, ci_upper)

  cat("Confidence Interval for alpha =", a, "\n")
  print(conf_int)
}
```

```
## Confidence Interval for alpha = 0.05
##                     ci_lower    ci_upper
## (Intercept)       1.3101230  1.64492318
## DirectionSouth   -0.1529131  0.12045010
## Sectionsection B -0.5579135 -0.12568833
## Sectionsection C -0.3163811  0.11584409
## Sectionsection D -0.3185987  0.11362651
## Sectionsection E -0.3703857  0.06183951
## Confidence Interval for alpha = 0.02
##                     ci_lower    ci_upper
## (Intercept)       1.2788302  1.67621594
## DirectionSouth   -0.1784635  0.14600054
## Sectionsection B -0.5983123 -0.08528954
## Sectionsection C -0.3567799  0.15624288
## Sectionsection D -0.3589975  0.15402530
## Sectionsection E -0.4107845  0.10223830
## Confidence Interval for alpha = 0.01
##                     ci_lower    ci_upper
## (Intercept)       1.2575221  1.69752410
## DirectionSouth   -0.1958616  0.16339858
## Sectionsection B -0.6258210 -0.05778083
## Sectionsection C -0.3842886  0.18375159
## Sectionsection D -0.3865062  0.18153401
## Sectionsection E -0.4382932  0.12974701
```

Plot bias

```r
library(ggplot2)

plot_data <- data.frame(
  index = 1:240,
  actual = data$ERROR,
  predicted = error_pred
)

ggplot(plot_data, aes(x = index)) +
  geom_line(aes(y = actual, color = "Actual")) +
  geom_line(aes(y = predicted, color = "Predicted")) +
  labs(title = "Comparison of Actual and Predicted Values of Error",
       x = "Index",
       y = "Error",
```

```
      color = "Legend") +
theme_minimal()
```

